

# When Listening Becomes Easier: Scrubbing Visual Cues for Shortcut-Free VLAs

Author(s)

**Abstract**—Shortcut learning is a prevalent issue in robot learning. The limited diversity of robot demonstration datasets can mislead policies into exploiting spurious correlations between tasks and irrelevant features, such as viewpoint or background. Collecting sufficiently diverse robot demonstrations is costly and inefficient, motivating algorithmic alternatives. We focus on vision-language-action (VLA) models and discover that different vision-language model backbones exhibit substantially different levels of susceptibility to visual shortcut learning. We find that model behavior correlates with our proposed representation-level metric, *action margin*, which requires no policy rollouts. Visual shortcuts consistently enter action representations in early layers, with models differing in the extent to which later layers correct them by incorporating language information. To boost models’ attention to language, we introduce task scrubbing, a new domain-adversarial training method that decreases models’ likelihood of using visual shortcuts and improves VLAs’ generalization. Experiments in both simulation and the real world across multiple VLAs and visual cues show that task scrubbing improves out-of-distribution robustness and often eliminates visual shortcut learning.

## I. INTRODUCTION

Vision-language-action (VLA) models are general-purpose robot policies built on pretrained vision-language models (VLMs) [alayrac2022flamingo](#), [liu2023visual](#) for their visual and linguistic abilities [brohan2023rt](#), [kim2024openvla](#). Although a large variety of different VLA models [black2024pi’0](#), [shukor2025smolvla](#), [zheng2026x](#), [wang2026qwen](#) have been proposed, they share a common design: a VLM encodes and processes visual and language information and produces representations which are then used to predict actions. Actions are predicted either by the same backbone [kim2024openvla](#), [belkhale2024minivla](#) or by a separate transformer, commonly called an action expert, that attends to the VLM representations. This design has produced policies that transfer pretrained vision-language knowledge to diverse manipulation tasks and generalize beyond individual training environments [brohan2023rt](#), [black2024pi’0](#), [intelligence2025pi](#). Its appeal rests on an implicit premise: the policy should inherit sufficiently strong visual-language understanding from the VLM to condition its actions on the language instruction.

However, in practice, this premise is less reliable than benchmark performance suggests [jiang2026we](#), [fei2025libero](#), [xu2026unmasking](#). VLAs remain brittle to visual perturbations and initial-state changes [fei2025libero](#), may fail to generalize across linguistic paraphrases [kim2026liberopara](#), and can lose useful pretrained structure while adapting their representations to precise spatial and action

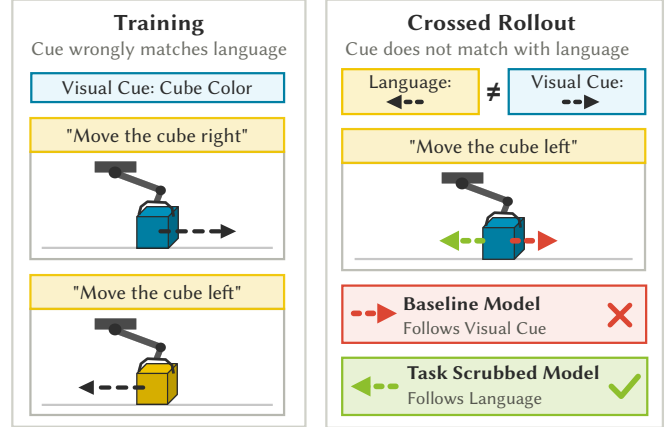


Fig. 1. **Visual shortcut-learning setup.** During training (left), the model sees two different tasks with visual cues that match the language instruction. When the model is then evaluated (right) out-of-distribution where the visual cue does not match the language, the baseline model follows the visual cue rather than the language. Applying task scrubbing during training encourages the policy to follow the instruction.

demands [driess2025knowledge](#), [grover2025enhancing](#), [dalal2026generalizable](#). More generally, robot policies tend to rely on irrelevant features captured in their training distribution [xing2025shortcut](#). For example, proxies such as viewpoint, background, object color, distractors, or proprioceptive state may provide a strong signal for task identity in the limited diversity of the training data [swann2026sparse](#), [grant2026not](#). This behavior is an instance of shortcut learning [geirhos2020shortcut](#), [xing2025shortcut](#). Models affected by shortcut learning achieve high in-distribution (ID) success but follow inconsequential cues when determining the task under out-of-distribution (OOD) conditions and thus fail.

We focus on *visual shortcuts* (??): task identity-correlated visual features that the policy uses as substitutes for the language instruction. These shortcuts are especially common as small visual variations between tasks can easily enter training demonstrations through shifts in camera position, background changes, or task setup. When this unwanted association is broken, a shortcut policy executes the visual cue-consistent behavior instead of the instructed behavior. Existing work largely describes this failure as a data-diversity problem [xing2025shortcut](#), without studying why different VLA architectures trained on the same data can have different shortcut propensities. It also remains unclear how shortcuts enter action computation or how the visual shortcut pathway can be suppressed without discarding useful visual

information.

To understand how model architectures influence shortcut susceptibility, we evaluate multiple existing VLAs and our own variants to better understand the effect of VLM choice. We find that shortcut susceptibility varies substantially across VLM backbones on the same dataset. Further, we introduce the action margin metric, which tracks whether a model is likely to shortcut by the alignment of the action-token hidden states with task- and cue-consistent reference states. Using this metric, we can directly track the development of shortcut actions during inference and find a mechanistic explanation. Attention knockout **geva2023dissecting** shows that task-correlated visual features shift action representations toward the shortcut in early layers with models differing in whether later computation restores alignment with the instruction. Based on these findings, we introduce task scrubbing to encourage models to attend to language information. Task scrubbing is an adversarial auxiliary loss that causes visual representations that predict task identity to shift during training, making them less reliable for action prediction. The intervention increases the relative influence of language when the model learns to predict actions, without removing information from the visual input.

Our contributions are:

- we show that shortcut susceptibility is strongly architecture dependent: VLA variants trained on the same data behave substantially differently, while VLM backbone size and general performance do not predict robustness;
- we introduce a systematic framework for analyzing shortcut behavior: action margin tracks behavioral shortcut reliance without rollouts, while attention knockout identifies how visual and language information shape action representations; and
- we introduce task scrubbing, a gradient-reversal method that makes task prediction from the visual pathway harder during training and causes policies to rely more strongly on language, thereby reducing shortcut learning.

## II. BACKGROUND AND RELATED WORK

### A. Vision-Language-Action Models

VLAs map visual observations, language instructions, and, in many systems, proprioceptive state to robot actions **brohan2023rt**, **kim2024openvla**. They vary in how they connect pretrained visual-language representations to control. Some models predict tokenized actions autoregressively with the VLM backbone itself **kim2024openvla**, **belkhale2024minivla**, whereas others pass image and text representations to a separate action expert that generates continuous action chunks **black2024pi<sup>0</sup>**, **shukor2025smolvla**. Multi-pathway systems such as  $\pi_{0.5}$  make this separation more explicit by assigning goal semantics and motor generation to different components **intelligence2025pi**. Across these designs, the expected benefit is that the VLM supplies broad semantic and visual abstractions while the action component adapts them to motor execution.

Despite their successes, VLAs face general robustness and representation-learning challenges. LIBERO-Plus **fei2025libero** documents brittleness under distractors, initial-state changes, and other visual variations, while LIBERO-Para shows that robustness to linguistic rephrasing cannot be assumed from VLM pretraining **kim2026liberopara**. VLM features may omit fine spatial or action-relevant details, motivating adaptation during policy training **driess2025knowledge**. However, broad fine-tuning can degrade pretrained visual-language structure and hurt generalization **grover2025enhancing**, **dalal2026generalizable**. Freezing the pretrained VLM to prevent information loss does not generally help and can reduce task performance without improving language generalization **kim2026liberopara**. VLAs therefore face a tension between adapting representations to predict actions accurately and preserving the general structure to enable generalization. Various approaches have been proposed to improve representations, such as using stronger VLMs **wu2026vlanext**, utilizing more informative visual embeddings **miao2026jepa**, or adding representation-preserving objectives **driess2025knowledge**, **grover2025enhancing**. Larger systems such as  $\pi_{0.7}$  combine broader data, model, and training changes to improve generalization **intelligence2026pi**. These results suggest that architecture and training affect representation quality and robustness.

### B. Shortcut Learning in Robot Policy Training

Shortcut learning occurs when a policy exploits a predictive correlation that is easier to learn than the intended task rule and consequently fails under distribution shift **geirhos2020shortcut**. In generalist robot policies, insufficient dataset diversity and fragmentation allow task identity to become entangled with particular scenes, embodiments, camera configurations, or trajectories **xing2025shortcut**. More data reduces this problem only when it introduces the relevant variation.

Mechanistic studies find weak prompt control when visual appearance already determines the behavior and identify internal features associated with memorized trajectories **grant2026not**, **swann2026sparse**. These findings challenge the assumption that a successful VLA necessarily learns a compositional mapping from instruction and scene to action. The model may instead interpolate among visually indexed motor programs.

Existing mitigations attempt to break shortcut correlations through broader data collection **xing2025shortcut**. Such interventions can be costly, benchmark-specific, and difficult to transfer. Removing shortcut-prone observation channels can also improve generalization **zhao2025you**, but permanently discarding an input removes information that other tasks may require. AC-VLA addresses visual shortcut reliance by selectively removing wrist-camera observations when the gripper is closed **peng2026ac**. DeLock takes an alternative approach of recovering the pretrained knowledge after fine-tuning using a contrastive prompt and steering **huang2026breaking**.

We expand on prior analysis and investigate how the model’s vision-language backbone impacts a policy’s propensity to follow visual shortcuts. Rather than permanently removing information or requiring additional data, we develop a training modification which causes VLAs to avoid relying on visual shortcuts.

### C. Adversarial Representation Learning

Domain-adversarial representation learning **ganin2016domain** is a general approach to make representations from different data sources indistinguishable to prevent models from improperly relying on them when making decisions. It relies on an auxiliary classifier to predict a selected attribute from a representation and reverses its gradient into the encoder **ganin2016domain**. The classifier learns to recover the attribute, while the encoder is trained to make that prediction difficult. This mechanism is commonly used to promote invariance to domains or nuisance variables, particularly in computer vision **yang2024limits**, **feng2026rap**, but has not yet been applied in related robotics settings. While making two sources of data completely indistinguishable can be very difficult, we find that continually shifting task-predictive visual representations during training can be sufficient to make the policy rely on the instruction rather than the visual shortcut.

## III. PROBLEM FORMULATION AND ANALYSIS

### A. Formal Definition of Shortcut Learning

We first establish a formal definition of visual shortcut learning. Let  $T$  denote the task specified by language,  $U$  denote the task-relevant visual information needed to execute the task,  $Z$  denote a task-irrelevant visual cue, and  $A$  denote the desired action. The complete visual observation is  $X = (U, Z)$ . In the training distribution, task and cue identity are statistically dependent such that  $I_{\text{train}}(T; Z) > 0$ . In our case, shortcut learning refers to cases where the learned policy mistakenly relies on  $Z$  rather than  $T$ .

The intended policy uses the instruction and task-relevant visual information:  $\pi_{\text{intended}}(A \mid U, T)$ . A shortcut policy instead allows the irrelevant cue to substitute for the instruction:  $\pi_{\text{shortcut}}(A \mid U, Z)$ . Its behavior depends strongly on  $Z$  and is comparatively invariant to  $T$ , even though  $Z$  is not causally relevant to the desired action.

Operationally, training contains correlated pairs such as  $(T_A, Z_A)$  and  $(T_B, Z_B)$ . In-distribution (ID) evaluation retains these observed combinations, whereas out-of-distribution (OOD) evaluation crosses them:

$$(T_A, Z_B), \quad (T_B, Z_A). \quad (1)$$

A robust policy follows the language-specified task under these crossings; a shortcut policy follows the visual cue.

We report ID success on the task-cue combinations present during training and OOD success on crossed combinations with success corresponding to the standard task completion criterion of the environment. We additionally report shortcut rate (SC), defined as the percentage of all OOD

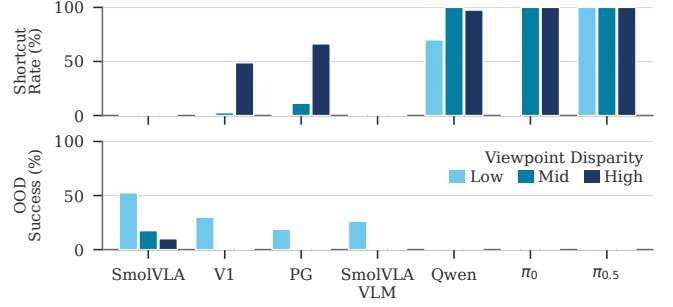


Fig. 2. OOD success and SC across VLA variants and low, medium, and high task-correlated viewpoint disparity on LIBERO Spatial.

episodes in which the policy attempts the cue-consistent ( $Z$ ) task rather than the instructed task ( $T$ ). An OOD episode cannot both succeed and shortcut.

### B. LIBERO Spatial Viewpoint-Disparity Setup

We adopt the controlled viewpoint-disparity setup of prior shortcut-learning work **xing2025shortcut**, where we have a dataset of two LIBERO Spatial **liu2023libero** tasks with task-specific viewpoints. The Low, Mid, and High conditions use progressively larger viewpoint differences between the two tasks. Across a broader set of VLAs, we reproduce the reported shortcut behavior (??). A rollout is considered to have used a shortcut if the gripper is closer to the incorrect bowl location when closing the first time. We compare finetuned  $\pi_0$  **black2024pi0**,  $\pi_{0.5}$  **intelligence2025pi**, SmolVLA **shukor2025smolvla**, and SmolVLA-VLM (SmolVLA with randomly initialized action-expert weights). We further introduce and evaluate novel SmolVLA variants, which use the same architecture but use a different VLM, to better understand the impact of the chosen backbone. SmolVLA-V1 replaces SmolVLA’s SmolVLM2 backbone with SmolVLM1, SmolVLA-PG uses the PaliGemma backbone (used by  $\pi_0$ ), and SmolVLA-Qwen uses Qwen2.5-VL-3B-Instruct **bai2025qwen25vltechnicalreport**.

Using the larger variety of models, we can better identify patterns between various VLM backbones. As shown in ??, SmolVLA and SmolVLA-VLM do not show any shortcut behavior, despite being significantly smaller models than  $\pi_0$  or  $\pi_{0.5}$ . The action pretraining further allows SmolVLA to achieve nonzero OOD success at all three disparity levels. Replacing its VLM backbone generally worsens performance, with the PaliGemma variant showing the strongest shortcut susceptibility. Further, while Qwen2.5-VL **bai2025qwen25vltechnicalreport** has the highest average benchmark performance of the evaluated VLMs, the resulting VLA has a low success rate and high SC. This indicates that general VLM performance does not predict the resulting VLA’s propensity for shortcut learning. The varying shortcut rates show that shortcut learning is not determined by the dataset alone and that both the VLM backbone and action pretraining matter.

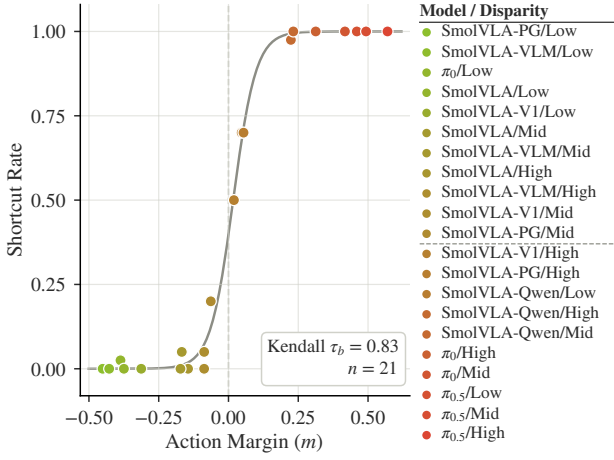


Fig. 3. Action margin shows a strong correlation with shortcut rate for the 21 model–disparity combinations. The representation-level metric strongly tracks shortcut reliance (Kendall’s  $\tau_b = 0.83$ ,  $p = 1.35 \times 10^{-7}$ ).

### C. Representation-Level Shortcut Magnitude

To estimate a model’s shortcut susceptibility without policy rollouts, we introduce action margin as a proxy, which measures whether action representations of crossed OOD states (??) align more closely with the instructed task or the visual cue. For each model, we use the average across the first observations of all training trajectories. We average the per-dimension normalized final-layer action-token hidden states  $h(o, \ell)$  for observation  $o$  under instruction  $\ell$ . We define the centroid representations  $c_{T,Z} = \mathbb{E}_{o \in O(Z)}[h(o, \ell_T)]$ , where  $T$  is the task and  $O(Z)$  is the set of visual observations which include visual cue  $Z$  for the two ID pairings ( $c_{T_A, Z_A}$  and  $c_{T_B, Z_B}$ ). We can then create an OOD sample such as  $x = (T_A, Z_B)$ . For this pairing,  $c_{\text{task}} = c_{T_A, Z_A}$  shares the instruction-specified task, whereas  $c_{\text{cue}} = c_{T_B, Z_B}$  shares the visual cue. Action margin  $m(x)$  is defined as the relative distances to these two representations and can be used to determine if the predicted action follows the task or the visual cue.

$$m(x) = \frac{\|h(x) - c_{\text{task}}\|_2 - \|h(x) - c_{\text{cue}}\|_2}{\|c_{\text{task}} - c_{\text{cue}}\|_2} \quad (2)$$

Positive values indicate that the representation is closer to the cue-matched centroid, while negative values indicate closer alignment to the instructed task.

We obtain a model-level value by averaging  $m(x)$  across the first observations of both crossed configurations. Restricting the analysis to the first step and fixing the flow-matching noise puts all trajectories at the same stage of motion and avoids confounding the representation with divergent rollout states. Across 21 model–viewpoint disparity combinations, action margin strongly correlates with SC (??), demonstrating that it is a good indicator of shortcut reliance across architectural variation.

### D. Mechanistic Analysis

Using action margin as a proxy for shortcut behavior, we apply attention knockouts **geva2023dissecting** to determine

which input streams causally move the action representation toward or away from the visual cue. At a selected layer, attention knockout masks all attention from action-token queries to one source—image, language, state, or other action tokens—before softmax renormalization. We compute the impact as  $\Delta m = m_{\text{full}} - m_{\text{knockout}}$  and average the behavior across ten initial trajectory frames. The results for all layers are plotted in ?? for  $\pi_0$ , SmolVLA-V1, and SmolVLA. A positive  $\Delta m$  indicates that removing a stream reduces cue alignment (the stream promotes the shortcut), whereas a negative  $\Delta m$  indicates that the stream actively corrects the shortcut.

?? shows that visual information injects the shortcut in early layers, consistently moving representations toward the cue. Later correction is model-dependent: language and action-token interactions can steer representations back toward the instructed task, but this correction is weak or even reversed in shortcut-prone models. Among the tested models, SmolVLA most effectively reverses early visual alignment via language features. These results motivate increasing the relative influence of language during training.

As the language correction is desirable, we track this influence using *language share*, which measures the relative extent to which language and visual cues steer the action representation along the task axis. As before, let  $h(o, \ell)$  denote the averaged final layer action-token representation, and let  $(T_A, Z_A)$  and  $(T_B, Z_B)$  denote the two ID combinations, with instructions  $\ell_{T_A}, \ell_{T_B}$ . We define the task axis as the normalized difference of the two ID centroids,

$$u = \frac{c_{T_A, Z_A} - c_{T_B, Z_B}}{\|c_{T_A, Z_A} - c_{T_B, Z_B}\|_2}, \quad (3)$$

and project two counterfactual displacements onto it. Re-running every observation under both instructions, with the image held fixed, gives the language steering

$$S_L = \left\langle \mathbb{E}_{o \in (O(Z_A) \cup O(Z_B))} [h(o, \ell_{T_A}) - h(o, \ell_{T_B})], u \right\rangle. \quad (4)$$

Holding the instruction fixed and contrasting the two visual cues gives the cue steering

$$S_V = \left\langle \mathbb{E}_{\ell \in \{\ell_{T_A}, \ell_{T_B}\}} \left[ \mathbb{E}_{O(Z_A)} [h(o, \ell)] - \mathbb{E}_{O(Z_B)} [h(o, \ell)] \right], u \right\rangle. \quad (5)$$

The diagnostic is their normalized balance,

$$\text{language share} = \frac{|S_L|}{|S_L| + |S_V|} \in [0, 1], \quad (6)$$

which rises toward 1 as language assumes control of the task axis and sits near 0 for a pure-shortcut policy. As in ??, the metric is computed on the first frame of each trajectory under fixed flow-matching noise, and allows us to track the model’s behavior during training.

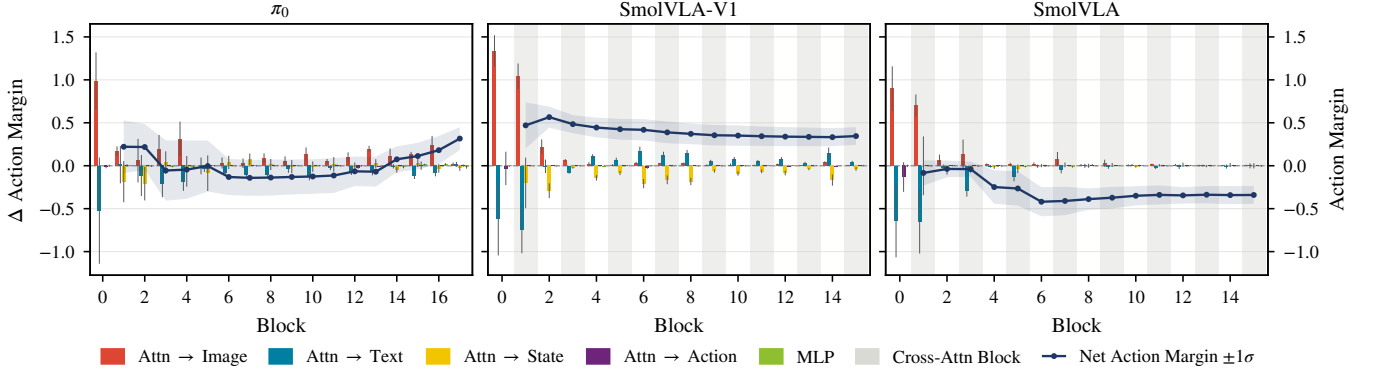


Fig. 4. Layer-wise attention-knockout effects for  $\pi_0$ , SmolVLA-V1, and SmolVLA. A positive value indicates that the removed attention promotes cue alignment, while negative values indicate it promotes task alignment. Removing visual attention generally shifts representations away from the cue, while language and action-token interactions can correct cue alignment in later layers.

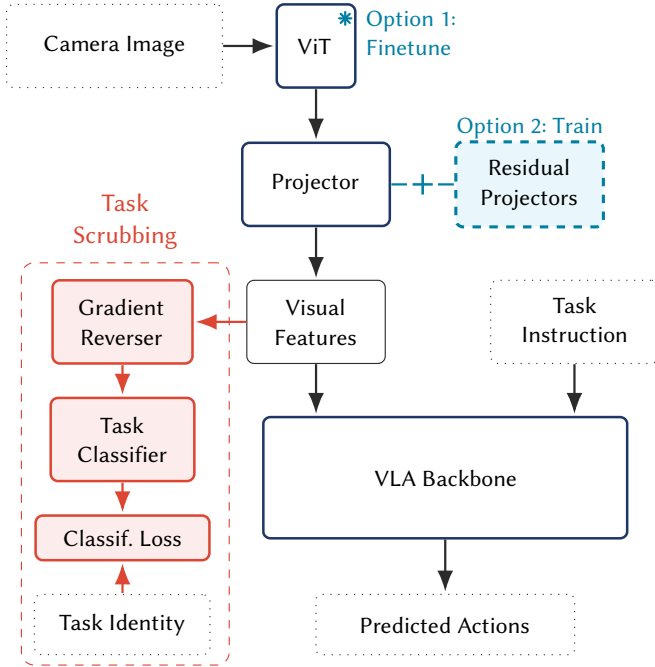


Fig. 5. Task scrubbing makes task information less available in the visual features during training, so that the VLA backbone learns to pay more attention to the language information. Red denotes the auxiliary task-scrubbing head used during training, which makes the visual pathways less informative about task identity. Either option shown in blue allows the visual pathway to be trainable with little additional cost.

#### IV. TASK SCRUBBING

Task scrubbing is a training-time intervention that makes visually encoded task identity less stable and thus less convenient for the model to exploit during policy training. As summarized in ??, it attaches an adversarial task classifier to the visual pathway: the classifier learns to recover task identity, while the visual pathway learns to frustrate that classifier. Because the resulting visual representations continually change, they provide the policy with a less stable signal for task identity. Language therefore becomes the more reliable task-identifying signal for the action objective. To summarize, the goal is not to remove all task-decodable

visual information, which is infeasible when task views differ strongly, but to prevent the task-irrelevant cues from becoming the easiest basis for action selection.

The classifier operates on the projected visual tokens before they enter the language-model or action backbone. Let  $v_1, \dots, v_N$  denote these tokens across all camera views. We first mean-pool them,  $\bar{v} = \frac{1}{N} \sum_{i=1}^N v_i$ , and then apply LayerNorm and a two-layer MLP with a GELU [hendrycks2016gelu](#) nonlinearity to predict the task label associated with the images. A gradient-reversal layer connects  $\bar{v}$  to this classifier.

The total training objective is

$$\mathcal{L} = \mathcal{L}_{\text{action}} + w\mathcal{L}_{\text{task}}, \quad (7)$$

where  $\mathcal{L}_{\text{task}}$  is cross-entropy on the task identity and  $w$  sets the relative scale of the auxiliary adversarial signal. This is a model- and setting-specific hyperparameter. In the forward pass, the classifier predicts task identity normally. During backpropagation, it minimizes  $\mathcal{L}_{\text{task}}$ , while gradient reversal causes the visual pathway to maximize the same objective. The classifier and policy are trained jointly from the beginning, and gradient reversal is applied throughout training.

For the auxiliary loss to affect the policy, the reversed gradient must reach trainable parameters in the visual pathway. When the ViT is fine-tuned, task scrubbing can update it directly. SmolVLA normally freezes both its VLM backbone and visual encoder, so we instead add a trainable residual adapter to the visual projection for such models. If the original projection is  $p(x)$ , the adapted output is

$$\tilde{p}(x) = p(x) + f_2(\text{GELU}(f_1(x))), \quad (8)$$

where  $f_1$  and  $f_2$  are trainable linear layers with  $f_2$  zero-initialized. Thus task scrubbing adds negligible computational overhead to training.

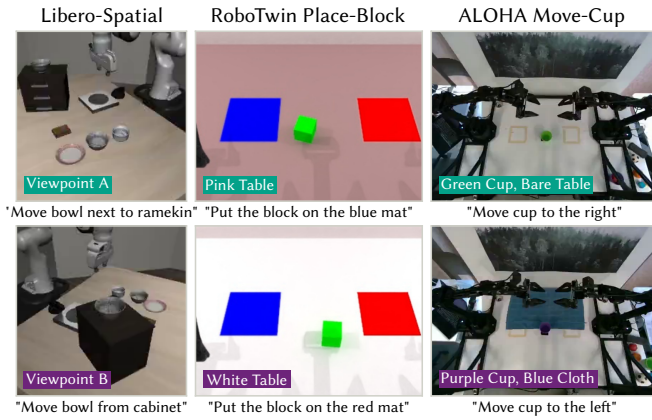


Fig. 6. Evaluation setup: The three different environments and task pairs used for evaluation.

## V. RESULTS

We evaluate task scrubbing on three separate experimental setups (??), involving three types of visual shortcut.

### A. Task Scrubbing Improves LIBERO Robustness

Returning to the setup from ??, we evaluate task scrubbing on the high-disparity LIBERO **liu2023libero** setting, where the two viewpoints differ by the largest rotation and the visual shortcut is the strongest. We compare the baseline policy with task scrubbing and report mean and sample standard deviations across three seeds. Each seed uses 40 rollouts for each of the four task-viewpoint configurations. ?? reports ID and OOD success and SC. The auxiliary-loss weight used for each model is reported in the Appendix.

Task scrubbing reduces SC and increases OOD success for  $\pi_0$ , the SmolVLA family, and MiniVLA **belkhale2024minivla**. For  $\pi_0$ , OOD success rises from 0% to 48% while SC falls from 99% to 0%, with no meaningful reduction in ID success. The main exception is  $\pi_{0.5}$ . Task scrubbing reduces mean SC and increases mean OOD success, but succeeds for only two of three seeds. The remaining seed retains an SC of approximately 90%. Mean ID success also falls from 66% to 45%. The SmolVLA variants show the strongest overall response: observed shortcut behavior is eliminated, and ID performance increases.

### B. Task Scrubbing Alters the Internal Information Pathway

During training, the accuracies of both the adversarial classifier and an additional gradient-isolated probe with the same architecture fall to chance or below. However, linear probes trained after policy training can still consistently decode task identity. Thus task scrubbing does not fully remove task information from the image stream. Instead the adversarial objective continually changes the task-relevant visual representation, providing the action learner with less stable visual signals for task identity. Language remains a direct and stable source of task identity. Consequently, task scrubbing changes which pathway is easiest to use rather than permanently deleting information from the image.

TABLE I  
LIBERO SPATIAL RESULTS UNDER HIGH VIEWPOINT DISPARITY.

| Model                                 | Condition | OOD Succ. (%) | SC (%)      | ID Succ. (%) |
|---------------------------------------|-----------|---------------|-------------|--------------|
| SmolVLA <b>shukor2025smolvla</b>      | Baseline  | 10 $\pm$ 1    | 0 $\pm$ 0   | 62 $\pm$ 3   |
|                                       | Scrubbed  | 73 $\pm$ 3    | 0 $\pm$ 0   | 74 $\pm$ 4   |
| SmolVLA-V1                            | Baseline  | 4 $\pm$ 6     | 45 $\pm$ 26 | 37 $\pm$ 4   |
|                                       | Scrubbed  | 64 $\pm$ 9    | 0 $\pm$ 0   | 66 $\pm$ 3   |
| SmolVLA-VLM                           | Baseline  | 4 $\pm$ 5     | 17 $\pm$ 28 | 34 $\pm$ 8   |
|                                       | Scrubbed  | 64 $\pm$ 10   | 0 $\pm$ 0   | 66 $\pm$ 10  |
| MiniVLA <b>belkhale2024minivla</b>    | Baseline  | 0 $\pm$ 1     | 100 $\pm$ 1 | 50 $\pm$ 5   |
|                                       | Scrubbed  | 26 $\pm$ 13   | 0 $\pm$ 0   | 44 $\pm$ 5   |
| $\pi_0$ <b>black2024pi0</b>           | Baseline  | 0 $\pm$ 0     | 99 $\pm$ 1  | 70 $\pm$ 9   |
|                                       | Scrubbed  | 48 $\pm$ 17   | 0 $\pm$ 0   | 69 $\pm$ 7   |
| $\pi_{0.5}$ <b>intelligence2025pi</b> | Baseline  | 0 $\pm$ 0     | 100 $\pm$ 0 | 66 $\pm$ 4   |
|                                       | Scrubbed  | 28 $\pm$ 25   | 28 $\pm$ 46 | 45 $\pm$ 4   |

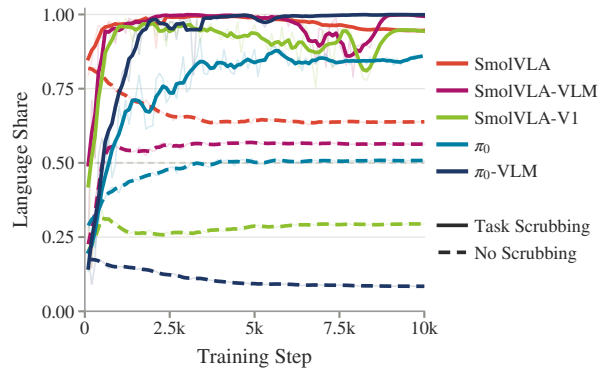


Fig. 7. Language share during training for SmolVLA, SmolVLA-VLM, SmolVLA-V1,  $\pi_0$ , and  $\pi_0$  without action pretraining. Task scrubbing increases the relative influence of language on the action representations. Action pretraining raises initial language share in both the SmolVLA and  $\pi_0$  comparisons, but pretrained models respond more slowly to scrubbing.

Training runs shown in ?? show this routing effect. Without task scrubbing, language share does not increase sufficiently or even decreases during training. With scrubbing, it rises sharply, indicating that action representations become more sensitive to language when determining the task information. Initial values differ across architectures: SmolVLA begins with a comparatively high language share, and action pretraining increases the initial value in both the SmolVLA and  $\pi_0$  comparisons, indicating that pretraining on a diverse set of tasks helps the models learn to use language.

### C. RoboTwin

We next evaluate whether the LIBERO results transfer to a different simulator and cue type. We construct a Place-Block task in the RoboTwin 2.0 **chen2025robotwin** simulator, which uses an ALOHA **Zhao-RSS-23** robot. The task requires grasping a cube at the center and placing it either on a blue mat to the left or on a red mat to the right using the corresponding arm according to the language instruction. During training, a brown background is paired with the left-placement task and a white background with the right-placement task. We train on a dataset of 400 episodes,

TABLE II  
PLACE-BLOCK RESULTS WITH DIFFERENT BACKGROUNDS.

| Model                                 | Condition | OOD Succ. (%) | SC (%)  | ID Succ. (%) | Method         | OOD Succ. (%) | SC (%)  | ID Succ. (%) |
|---------------------------------------|-----------|---------------|---------|--------------|----------------|---------------|---------|--------------|
| SmolVLA <b>shukor2025smolvla</b>      | Baseline  | 5 ± 2         | 0 ± 0   | 98 ± 3       | Scrubbed       | 73 ± 3        | 0 ± 0   | 74 ± 4       |
|                                       | Scrubbed  | 86 ± 3        | 0 ± 0   | 98 ± 3       | Attention Reg. | 67 ± 4        | 0 ± 0   | 67 ± 3       |
| SmolVLA-V1                            | Baseline  | 0 ± 0         | 100 ± 0 | 90 ± 11      | Scrubbed       | 64 ± 9        | 0 ± 0   | 66 ± 3       |
|                                       | Scrubbed  | 83 ± 6        | 0 ± 0   | 80 ± 5       | Attention Reg. | 56 ± 5        | 0 ± 0   | 59 ± 4       |
| SmolVLA-VLM                           | Baseline  | 0 ± 0         | 100 ± 0 | 90 ± 11      | Scrubbed       | 48 ± 17       | 0 ± 0   | 69 ± 7       |
|                                       | Scrubbed  | 45 ± 15       | 0 ± 0   | 50 ± 11      | Attention Reg. | 3 ± 2         | 68 ± 31 | 57 ± 8       |
| $\pi_0$ <b>black2024pi'0</b>          | Baseline  | 0 ± 0         | 100 ± 0 | 94 ± 1       | Scrubbed       | 28 ± 25       | 28 ± 46 | 45 ± 4       |
|                                       | Scrubbed  | 62 ± 12       | 0 ± 0   | 85 ± 0       | Attention Reg. | 0 ± 0         | 79 ± 10 | 50 ± 10      |
| $\pi_{0.5}$ <b>intelligence2025pi</b> | Baseline  | 0 ± 0         | 100 ± 0 | 97 ± 1       |                |               |         |              |
|                                       | Scrubbed  | 39 ± 8        | 39 ± 9  | 57 ± 1       |                |               |         |              |

TABLE III  
REAL-WORLD ALOHA PERFORMANCE.

| Cue          | Model                            | Scrub.? | OOD Succ. (%) | SC (%) | ID Succ. (%) |
|--------------|----------------------------------|---------|---------------|--------|--------------|
| Cup color    | SmolVLA <b>shukor2025smolvla</b> | No      | 77.5          | 0      | 95.0         |
|              |                                  | Yes     | 95.0          | 2.5    | 95.0         |
|              | SmolVLA-V1                       | No      | 57.5          | 17.5   | 95.0         |
|              |                                  | Yes     | 87.5          | 0      | 90.0         |
| +Back-ground | SmolVLA <b>shukor2025smolvla</b> | No      | 47.5          | 27.5   | 100.0        |
|              |                                  | Yes     | 92.5          | 0      | 97.5         |
|              | SmolVLA-V1                       | No      | 0             | 100.0  | 100.0        |
|              |                                  | Yes     | 100.0         | 0      | 97.5         |

200 of each task, generated using RoboTwin’s data generator.

Task success is measured by whether the cube is fully inside the instructed region. SC is measured by whether the gripper which closes closest to the cube is associated with the cue-consistent task. We evaluate each of the four cube-background color combinations 20 times and report standard deviations across three seeds. ?? shows the same overall pattern as LIBERO (see Appendix for the loss weights used). We do not evaluate MiniVLA as its ID performance is too low to support a meaningful comparison. SmolVLA-VLM has a similar pattern to  $\pi_{0.5}$  on LIBERO Spatial, where one seed performs significantly worse than the other two. Models more often see a drop in ID performance, possibly as this cue is stronger than the viewpoint change. But task scrubbing still decreases SC and boosts OOD success.

#### D. Real-Robot Experiments

Finally, we evaluate task scrubbing on a physical ALOHA **Zhao-RSS-23** platform controlled through the Trossen LeRobot setup. The robot must place a cup on either the left or right target according to a language instruction. We test the models across two different visual shortcut conditions. The first condition uses the cup color as a local cue. During training, a green cup is paired with the right target and a purple cup with the left target. The second condition introduces an additional stronger global cue: the purple-left demonstrations also contain a blue towel in the

TABLE IV  
ATTENTION REGULARIZATION VERSUS TASK SCRUBBING.

background, while the green-right demonstrations use the normal table background. This condition therefore correlates task identity with both cup color and background. SC is recorded when the gripper associated with the cue-consistent task closes in an attempt to grasp the cup.

We collect 100 demonstrations per task and evaluate one trained seed per condition with twenty trials per configuration. ?? shows that task scrubbing improves both models under both cue conditions. We use the same weights as in the LIBERO Spatial results (see Appendix). With cup color alone, little shortcut learning occurs but models’ OOD success still benefits from task scrubbing. In the combined background-and-color condition, task scrubbing eliminates all observed shortcut episodes and yields OOD success rates above 90% for both models. These results show that the intervention transfers to physical control and is effective against both local and global visual cues.

#### E. Ablation: Direct Attention Regularization

Because task scrubbing increases language share, an alternative is to manipulate attention directly to produce a similar effect. To the action loss, we add an attention regularizer equal to the mean of the action-to-image attention over all heads and layers containing cross-modal attention:

$$\mathcal{L}_{\text{attn}} = \lambda \text{mean}(A_{\text{action} \rightarrow \text{image}}), \quad \lambda = 1. \quad (9)$$

Minimizing this term penalizes action-to-image attention relative to the available attention budget and drives language share to above 90% in all evaluated models.

We compare attention regularization with task scrubbing on high-disparity LIBERO. ?? reports means and standard deviations. Attention regularization works for the SmolVLA variants, but trails task scrubbing in success rates. It fails for the  $\pi$  family:  $\pi_0$  retains 68% SC and  $\pi_{0.5}$  retains 79% SC. Thus, forcing a high language attention ratio does not guarantee that language controls the action as attention amounts are an incomplete measure of information flow.

## VI. DISCUSSION

Our results show that VLM choice changes how readily task-correlated visual information controls a VLA’s action component. Models trained on the same demonstrations can differ substantially in shortcut behavior, and replacing SmolVLA’s backbone is sufficient to change its robustness. Shortcut learning should therefore not be treated solely as

a dataset property. Architecture, pretraining, and the ability of downstream layers to use language to correct visually induced cue alignment are also important.

The mechanistic analysis further shows that robustness does not require a model to stop encoding the cue. Visual task identity remains decodable after task scrubbing. What matters is whether that information dominates action computation and whether later layers use language information to restore task alignment. Task scrubbing changes this competition by making visually encoded task identity unstable during training. The resulting increase in language share is evidence of rerouting toward language, rather than complete removal of visual information.

The proposed diagnostics may also support model selection and training monitoring. Under controlled crossings, the action margin can identify backbones whose action representations are likely to follow the cue. Language share can track whether task scrubbing shifts representational influence toward language during training and may provide a signal for adjusting  $w$ .

## VII. CONCLUSION AND LIMITATIONS

Using action margin together with attention knockout, we show that visual cues shift early-layer VLA action representations toward cue-consistent behavior. We observe differences in how later computation reflects language control across VLM backbones, which can be tracked during training using language share. Task scrubbing makes visually encoded task identity unstable during training, causing the policy to prefer the language instruction as its task-identifying signal. Across LIBERO, RoboTwin, and real-robot experiments, it reduces shortcut behavior and improves OOD success. Overall, our findings establish that directly manipulating information routing is an effective way to prevent visual shortcut learning in VLAs.

Nevertheless, our approach still has limitations. Task scrubbing requires task labels and a visual pathway with trainable parameters. Frozen architectures therefore require an adapter. It also requires architecture-specific calibration of the auxiliary adversarial signal; developing a reliable ID-only calibration criterion remains open. Our mechanistic analysis focuses on VLM-plus-action-expert architectures, and MiniVLA is consequently included only in the behavioral intervention results, as supporting its autoregressive nature would require architecture-specific methodology. Our evaluations are limited to settings containing two paired tasks and do not investigate how task scrubbing behaves in multi-task settings. Finally, some seeds retain shortcut behavior, and task scrubbing can reduce ID performance.

## APPENDIX

The task scrub scale  $w$  was selected for each architecture and task. Using a single seed, we select a scale that reduces SC and maximizes OOD success without reducing ID success when possible. ?? shows the values used.

TABLE V  
SELECTED AUXILIARY-LOSS SCALES FOR TASK-SCRUBBING.

| Model                                 | LIBERO | RoboTwin | Real robot |
|---------------------------------------|--------|----------|------------|
| SmolVLA <b>shukor2025smolvla</b>      | 2      | 0.75     | 2          |
| SmolVLA-V1                            | 1      | 0.175    | 1          |
| SmolVLA-VLM                           | 2      | 0.25     | –          |
| MiniVLA <b>belkhale2024minivla</b>    | 12     | –        | –          |
| $\pi_0$ <b>black2024pi'0</b>          | 0.5    | 0.35     | –          |
| $\pi_{0.5}$ <b>intelligence2025pi</b> | 0.8    | 0.55     | –          |

## ACKNOWLEDGMENTS

AI-assisted coding tools, including agents and coding harnesses, were used during development of the code.